

Rajshahi University of Engineering & Technology

Rajshahi—6204, Bangladesh

Department of Computer Science & Engineering

Project Design Report

CSE 3206 — Software Engineering Sessional

Lab 2: Software Process Models, Requirement Analysis & MVP Development

Course Outcome (CO2): Design and implement software solutions that effectively address user requirements and enhance system reliability and performance.

Program Outcome (PO3): Design and Development of Software Solutions

QuickPOS

A Super Shop Point of Sale (POS) System

Minimum Viable Product (MVP) — Increment 4 of 4

Selected Software Process Model

Incremental Development

The system is delivered as a sequence of small, independently demonstrable increments, each merged into `main` through a reviewed pull request.

Submission Details

Group

Group #08 — Section A (2nd 30)

Submitted by

Tanzirul Islam • Shuvo (Mia Shuvo) • Ruan (Mahdi Hosen Ruan)

Submitted to

Course Teacher, Department of Computer Science & Engineering, RUET

GitHub repository

<https://github.com/TanzirulIslam22/QuickPOS>

Project Design Report

docs/Requirement_Report.pdf (source:
docs/Project_Design_Report.tex)

Date of submission

01 October 2026

Lab 2 deliverables: GitHub repository (branches, pull requests, commit history) • Project Design Report • MVP demonstration • Individual viva

Table of Contents

Contents

1 Team Information and Work Ownership	4
Team Information and Work Ownership	4
1.1 Team members, roles and branches	4
1.2 Ownership map	4
1.3 Collaboration rules followed by the team	5
2 Project Title and Overview	5
Project Title and Overview	5
3 Problem Statement	5
4 Project Objectives	6
5 Stakeholder Analysis	6
6 Scope of the Project	7
6.1 In scope for this lab	7
6.2 Explicitly out of scope for the MVP	7
7 Functional Requirements	7
8 Non-Functional Requirements	8
9 User Stories and Use Cases	9
10 Assumptions and Constraints	11
10.1 Assumptions	11
10.2 Constraints	11
11 Selected Software Process Model: Incremental Development	11
11.1 How the model is applied	12
11.2 Increment plan and delivery record	12
11.3 Honest statement of Increment 2	13
11.4 Figure: incremental delivery for QuickPOS	13
12 Justification of the Selected Process Model	13
13 Comparison with Alternative Process Models	14
13.1 Comparative summary	14
13.2 Why Waterfall was less suitable	14
13.3 Why Spiral was less suitable	15
13.4 Why Agile Scrum was less suitable	15
13.5 The remaining models	15

14 MVP Design Overview	15
14.1 Technology stack	15
14.2 Architecture	16
14.3 Backend interface	16
14.4 Module breakdown and ownership	17
14.5 Component structure	18
14.6 Cash handling as a transaction	18
14.7 Sale document	18
14.8 MVP demonstration screens	19
14.9 Backlog carried to the next increment	21
15 GitHub Collaboration Evidence	21
15.1 Repository	21
15.2 Repository directory structure	21
15.3 Branching strategy	22
15.4 Pull requests	23
15.5 Commit history	23
15.6 Pull request review protocol	23
15.7 Collaboration issues encountered	24
16 Challenges Encountered and Engineering Mitigations	24
16.1 C-1: The first report build rendered faint, low-contrast text	24
16.2 C-2: Client-side payment validation is not enforcement	24
16.3 C-3: A sale could have partially decremented stock	24
16.4 C-4: An increment was merged with less delivered than its branch name promised	24
16.5 C-5: Environment differences between machines	25
16.6 C-6: The billing component was growing beyond a single responsibility	25
17 Conclusion	25

1 Team Information and Work Ownership

This section is the single authoritative record of *who contributed which part of the system*. It is reproduced in the repository (`README.md` and `docs/TEAM_WORKFLOW.md`) so that the contribution split can be verified against the Git commit history rather than taken on trust.

1.1 Team members, roles and branches

Table: Member, Git identity, feature branch and module responsibility				
Team Member	GitHub Author / As	Role	Git Branch	Module Responsibility
Tanzirul Islam	TanzirulIslam22 tanzirul.islam56@gmail.com	Team Lead & Backend / Core Developer	tanzirul/mvp-base tanzirul/report-lab2	Repository and GitHub workflow setup; Express server, MongoDB models and REST routes; React application shell and session gate; Login, Products and POS pages; cash handling and stock deduction; sales history; report and documentation
Shuvo	Hi-There-shuvo saddamshuvo1710@gmail.com	Frontend Feature Developer	shuvo/category-receipt	Printable invoice / receipt module (<code>Invoice.jsx</code>) and its integration into the checkout success path in <code>POS.jsx</code> , including the print stylesheet
Ruan	mahdi-ruan mhr359359@gmail.com	Frontend Feature Developer	ruan/dashboard	Planned sales summary dashboard increment (PR #2); delivered increment contains route-level hardening of <code>saleRoutes.js</code> ; outstanding summary widget carried to the backlog as B-01

1.2 Ownership map

Figure 1 maps each member to the part of the system that member owns. The same mapping is applied file by file in the module breakdown of Section 14.

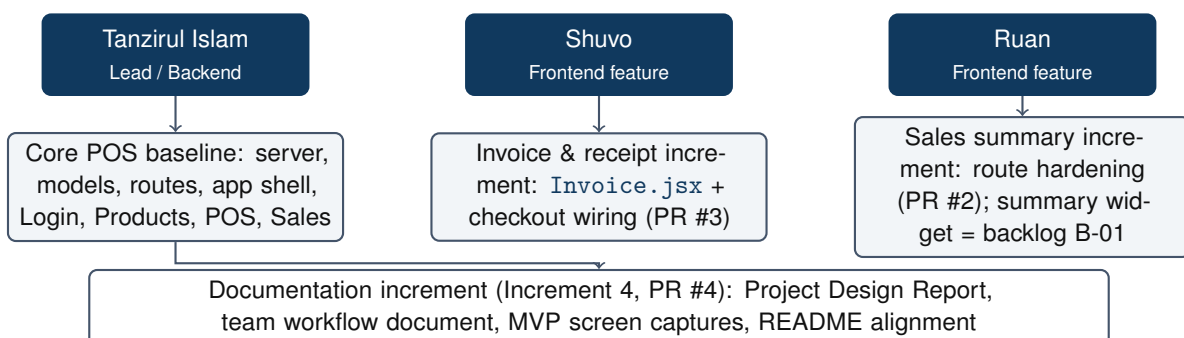


Figure 1: Work ownership: member → feature branch → module.

1.3 Collaboration rules followed by the team

- `main` is the only integration branch; no member pushes directly to it.
- Every increment is developed on its own `member/feature` branch, committed with a meaningful message, pushed, and submitted as a pull request against `main`.
- A pull request is reviewed by at least one other member before it is merged.
- Each pull request states, in its description, which requirements it satisfies and which requirements remain open. Where an increment is closed with work still outstanding, the gap is recorded as a numbered backlog item instead of being left implicit (see Section 14.9).

2 Project Title and Overview

QuickPOS — A Super Shop Point of Sale (POS) System

QuickPOS is a web-based billing and stock system for a small neighbourhood super shop. It replaces the shop's paper bills and stock notebooks with a single screen on which a cashier logs in, builds a bill, takes the customer's cash, and is told exactly how much change to return — while every sale is written to a database and the stock count of each sold item is reduced automatically.

QuickPOS is this lab's Minimum Viable Product (MVP): a small but complete and functional slice of a larger system. It deliberately contains only what is needed to demonstrate that the core billing concept is technically feasible — authentication, one core domain (the product-and-sale domain), basic persistence and a simple user interface — and nothing else. The code is written as a set of small, replaceable modules so that the following labs of the semester (design patterns, testing) can build on it without a rewrite.

3 Problem Statement

A single-bay super shop in Bangladesh typically records its sales on paper bills kept in a box, and tracks remaining stock in a separate notebook. The result is four recurring, concrete problems:

- **Slow and error-prone billing.** Every sale requires the cashier to look up a price in the notebook and write the bill by hand. At peak hours a queue forms, and arithmetic mistakes in the total or in the change are common.
- **No trustworthy stock count.** Because the notebook is updated by hand and only at the shop owner's discretion, the recorded stock routinely disagrees with the shelf. The owner then either sells an item that is not there, or discards items that were actually available.
- **No record of cash movement.** Change given to a customer is calculated mentally. When the drawer does not balance at the end of the day there is no way to determine which bill was miscalculated, so daily takings cannot be verified.
- **No sales history for decisions.** Because bills are paper, the owner cannot answer simple questions such as which product sells most, what the daily revenue was, or which items need restocking.

QuickPOS addresses each of these directly: an itemised billing screen that computes the total, an explicit cash-handling panel that computes and validates change before a sale can be committed, automatic stock deduction that refuses a sale exceeding available stock, and a persistent sales history that can be reviewed at any time.

4 Project Objectives

1. Provide a fast, single-screen billing interface that a new cashier can operate without training.
2. Maintain accurate product information — name, price, stock and category — in one place, with the catalogue maintained through the application itself.
3. Compute the bill total automatically from the cart so that no arithmetic is done by hand at the counter.
4. Record the cash actually received from the customer, compute the change to return, and *reject* the sale whenever the cash received is less than the total.
5. Reduce product stock automatically as part of the checkout transaction, and refuse any sale that would drive stock below zero.
6. Persist every completed sale — its items, total, cash paid, change and timestamp — and display the recent sales history.
7. Produce a printable invoice for each sale so the customer leaves with a receipt.
8. Gate the whole system behind a staff login so that the billing screen is not open to walk-in customers.
9. Ship demonstrable sample data so that the MVP can be reviewed immediately after installation, without manual database preparation.
10. Demonstrate a functioning Incremental Development process executed by a three-member team through feature branches and reviewed pull requests.

5 Stakeholder Analysis

Table: Stakeholder, interest or need, and primary interactions with the system

Stakeholder	Interest / Need	Primary Interactions
Shop Owner	Daily revenue figures, control over the product catalogue and stock levels, and a low-cost tool that does not need an IT background to operate	Reviews the sales history; adds, edits and removes products; monitors stock after every checkout
Cashier	A fast billing screen that is easy to learn, that never lets an incorrect total reach the customer, and that always states the change to return	Logs in, builds a cart, enters the cash received, completes the checkout, prints the invoice
Customer	An accurate total, the correct change, and a receipt as proof of purchase	Interacts indirectly: is served by the cashier's billing screen; receives the printed invoice
System Administrator	Control of the product data and of who may use the system	Manages products, resets the session, seeds demonstration data
Regulator / Auditor (indirect)	Traceability of individual sales transactions	Would consume the persisted sale records (date, items, total, cash paid, change) exported from the sales history

All stakeholders identified above expect the system to be fast, reliable and simple to use. These three expectations are the origin of most of the non-functional requirements in Section 8.

6 Scope of the Project

6.1 In scope for this lab

- Staff authentication and session handling for a single shop terminal.
- Product catalogue with list, add and delete operations.
- Billing: cart with quantities, automatic total, cash-entered payment, change calculation and a printable invoice.
- Automatic stock validation and deduction at checkout.
- Persistent sales history.
- Demonstration (seed) data.

6.2 Explicitly out of scope for the MVP

Supplier and purchase records, bar-code scanning, multi-branch stock transfer, customer loyalty accounts, employee salary calculation, taxation, hardware receipt printers, offline mode, and internet-facing deployment. Recording what is *not* being built is part of keeping an MVP an MVP; the excluded items are carried in the backlog of Section 14.9 for later increments.

7 Functional Requirements

The requirements below are traceable to the delivered code. The *Implemented in* column states where each requirement is satisfied and which team member's increment delivered it. Requirements FR-01 to FR-18 are the MVP requirement set; FR-06 is available at the API level and is queued for the user interface in backlog item B-02.

Table 1: Functional requirements with implementation traceability.

Table: Functional requirements and where each is implemented			
ID	Status	Requirement	Implemented in
FR-01	Done	The system shall authenticate a staff member with a username and a password before any billing screen is shown.	<code>Login.jsx</code> ; credential pair admin / 1234
FR-02	Done	The system shall keep the authenticated session across page reloads and shall end it on logout, returning the user to the login screen.	<code>App.jsx</code> via <code>localStorage</code>
FR-03	Done	The system shall list every product with its name, unit price and current stock level.	<code>POS.jsx</code> , <code>Products.jsx</code> ; GET <code>/api/products</code>
FR-04	Done	The system shall allow a new product to be added with name, price and stock.	<code>Products.jsx</code> ; POST <code>/api/products</code>
FR-05	Done	The system shall allow an existing product to be deleted.	<code>Products.jsx</code> ; DELETE <code>/api/products/:id</code>
FR-06	API only	The system shall allow an existing product's details to be updated.	PUT <code>/api/products/:id</code> implemented in <code>productRoutes.js</code> ; no user interface yet (backlog B-02)
FR-07	Done	The system shall allow a product to be added to the bill, and repeated additions of the same product shall increase its quantity.	<code>POS.jsx</code> <code>add()</code> function

Functional requirements (continued)			
FR-08	Done	The system shall allow the cashier to empty the current bill without starting a session again.	<code>POS.jsx</code> <i>Clear</i> button
FR-09	Done	The system shall compute the running bill total from the cart contents.	<code>POS.jsx</code> <code>total</code> reducer
FR-10	Done	The system shall accept the amount of cash handed over by the customer.	<code>POS.jsx</code> <code>paidAmount</code> input, shown once the cart is non-empty
FR-11	Done	The system shall compute the change to return as (cash received – bill total) and display it live while the cashier types.	<code>POS.jsx</code> <code>change</code> value and <i>Change to Return</i> panel
FR-12	Done	The system shall refuse to complete a sale when the cash received is less than the bill total, both in the user interface and on the server.	<code>POS.jsx</code> disables <i>Checkout</i> ; <code>saleRoutes.js</code> returns HTTP 400
FR-13	Done	On a successful sale the system shall reduce the stock of every sold item by the quantity sold.	<code>saleRoutes.js</code> <code>p.stock -= it.qty</code>
FR-14	Done	The system shall refuse a sale for any item whose requested quantity exceeds the available stock.	<code>saleRoutes.js</code> stock check before any write
FR-15	Done	The system shall store every completed sale with its itemised lines, total, cash received, change and timestamp.	<code>Sale.js</code> model; POST <code>/api/sales</code>
FR-16	Done	The system shall display the most recent sales, newest first, with each bill's total, cash paid and change.	<code>Sales.jsx</code> ; GET <code>/api/sales</code> (latest 50)
FR-17	Done	After a successful sale the system shall present a printable invoice listing every line, the subtotal, the cash received, the change returned and the grand total.	<code>Invoice.jsx</code> ; <code>window.print()</code> with an <code>@media print</code> stylesheet
FR-18	Done	The system shall be able to populate the catalogue with demonstration products so the MVP can be demonstrated immediately.	<code>server.js</code> GET <code>/api/seed</code>

8 Non-Functional Requirements

Table 2: Non-functional requirements.

Table: Non-functional requirements		
ID	Category	Requirement
NFR-01	Performance	A checkout transaction shall complete within 2 seconds on the shop's local network, excluding the cashier's typing time.
NFR-02	Performance	Re-opening the product list after a sale shall not require a full page reload; the billing screen shall refresh its stock view after each checkout.

Non-functional requirements (continued)		
NFR-03	Usability	The complete billing task shall be performable from a single screen using only the keyboard and one pointing device, with no training manual and no multi-page wizard.
NFR-04	Usability	Destructive and irreversible actions (completing a sale, clearing a bill) shall be visually distinguishable from routine actions.
NFR-05	Availability	The system shall run on a single computer inside the shop and shall continue to work with no internet connection.
NFR-06	Portability	The system shall run on any operating system through a current desktop web browser, with no platform-specific component.
NFR-07	Maintainability	Source shall be separated into a frontend module, a backend module, a route layer and a data-model layer; each page and each route shall be independently replaceable.
NFR-08	Maintainability	All backend calls from the frontend shall pass through one helper module so that the endpoint contract is defined in exactly one place.
NFR-09	Security	No billing, catalogue or sales screen shall be reachable without an authenticated session.
NFR-10	Security / Integrity	Payment validation shall be repeated on the server; the client-side check shall be treated as a usability aid only, never as the enforcement point.
NFR-11	Reliability	A sale shall be written to the database before the billing screen is cleared, so that an interrupted session cannot lose a completed transaction.
NFR-12	Data Integrity	Stock shall never be recorded as negative; a sale exceeding available stock shall be rejected as a whole, without partially applying it.
NFR-13	Scalability	The product catalogue shall remain operable at 10 000 product records without a change in the interaction model.
NFR-14	Traceability	Each sale shall record its timestamp so that the day's takings can be reconstructed after the fact.
NFR-15	Printability	The invoice shall have a dedicated print stylesheet so that on-screen controls (buttons, backdrop) do not appear on the printed receipt.

9 User Stories and Use Cases

Six user stories are given below, each with an acceptance criterion written in Given/When/Then form. The acceptance criteria are the definition of “done” for the increment that delivers the story.

US-1 Staff authentication (Cashier)

As a cashier, I want to log in with my staff credentials so that the billing screen is not open to any customer or visitor.

Acceptance criteria:

- **Given** the application is opened with no active session, **when** the cashier submits `admin / 1234`, **then** the billing screen is shown and the session survives a page reload.
- **Given** the application is opened with no active session, **when** any other credential pair is submitted, **then** the login screen remains and no billing data is displayed.
- **Given** an active session, **when** *Logout* is pressed, **then** the session is destroyed and the login screen is shown again.

US-2 Building a bill (Cashier)

As a cashier, I want to add products to a bill and see the running total so that I can serve a customer quickly without doing arithmetic myself.

Acceptance criteria:

- **Given** the billing screen, **when** the cashier presses *Add* on a product, **then** that product appears in the bill with quantity 1 and the total increases by the unit price.
- **Given** a product is already in the bill, **when** *Add* is pressed again, **then** only its quantity increases and no duplicate line is created.
- **Given** a bill with three lines, **when** the cashier inspects the header, **then** the displayed total equals the sum of $\text{price} \times \text{quantity}$ over all three lines.

US-3 Cash handling and change (Cashier)

As a cashier, I want to type in the cash the customer handed over and be shown the change, so that I return the correct amount and never have to calculate it mentally.

Acceptance criteria:

- **Given** a bill of 490 Tk, **when** the cashier enters 500 Tk, **then** the panel shows *Change to Return: 10 Tk* and the *Checkout* button becomes enabled.
- **Given** a bill of 490 Tk, **when** the cashier enters 400 Tk, **then** the panel marks the amount as *insufficient*, the *Checkout* button stays disabled, and no sale is created.
- **Given** an enabled *Checkout* button, **when** it is pressed, **then** the API rejects the request with HTTP 400 if, and only if, the server also computes the payment as insufficient.

US-4 Automatic stock control (Shop Owner)

As a shop owner, I want the stock of an item to fall automatically when it is sold, so that the system never sells something the shelf does not have.

Acceptance criteria:

- **Given** an item with 100 units in stock, **when** a bill of 2 units is completed, **then** the stored stock becomes 98.
- **Given** an item with 1 unit in stock, **when** a bill of 5 units is submitted, **then** the whole sale is rejected and the stored stock is left unchanged at 1.

US-5 Printable invoice (Cashier & Customer)

As a cashier, I want a printable invoice to appear automatically after a sale so that the customer receives a receipt and I have proof of the transaction.

Acceptance criteria:

- **Given** a completed checkout, **when** the invoice appears, **then** it lists every sold line with quantity, unit price and line amount, plus the subtotal, the cash received, the change returned and the grand total.
- **Given** the invoice is on screen, **when** *Print Invoice* is pressed, **then** the browser print dialog opens and the printed page contains the receipt only — no buttons and no dimmed backdrop.

US-6 Sales history (Shop Owner)

As a shop owner, I want to see previous bills with their total, cash paid and change, so that I can check the day's takings.

Acceptance criteria:

- **Given** at least one completed sale, **when** the *Sales* screen is opened, **then** the bills are listed newest first, each with its timestamp, itemised lines, total, cash paid and change.
- **Given** no sale has been recorded, **when** the *Sales* screen is opened, **then** an explicit *No sales yet* message is shown instead of an empty page.

10 Assumptions and Constraints

10.1 Assumptions

- The shop operates a single billing terminal, so there is no requirement for concurrent cashiers or multi-session stock locking.
- All staff are trusted employees of one shop, so a single shared staff account is acceptable for the MVP; differentiated roles are deferred (backlog B-06).
- Prices do not change during an open bill, so the price captured at checkout is the authoritative price for that sale.
- The shop can run a small always-on computer with Node.js and a local MongoDB instance; cloud hosting is not required.
- Electrical power and network stability inside the shop are the shop's existing responsibility.

10.2 Constraints

- **Time.** The project is a single laboratory session of the first milestone of a semester-long project, developed by a three-member student team.
- **Scope.** The lab explicitly requires a Minimum Viable Product only; feature count is not the measure of success, engineering method is.
- **Process.** Every member must contribute through Git using a feature branch and a pull request; direct pushes to `main` are not permitted by the team's own working agreement.
- **Future labs.** The code must remain modular, because later labs will add design patterns and automated tests on top of this codebase.
- **Tooling.** Only freely available, quickly learnable technology may be used within the time available, which excludes heavyweight enterprise frameworks.

11 Selected Software Process Model: Incremental Development

The team selected the **Incremental Development** process model. QuickPOS is delivered as a sequence of increments. Each increment adds a coherent slice of functionality to a system that is already running, and each increment is integrated into the main branch through a reviewed pull request, so that at the end of every increment there is a *working, demonstrable* product rather than a partial one.

This choice also matches the process model suggested for this project in the lab assignment (*Super Shop POS System* → *Incremental*), but the selection below is justified on the project's own characteristics rather than on the suggestion alone (Section 12).

11.1 How the model is applied

- **Requirements are decomposed, not frozen.** The requirement set of Section 7 is partitioned into increments that each deliver an end-to-end slice — for example “cart → total → cash → change” is one slice, not four separate features.
- **Each increment is integrated.** Work happens on a feature branch and reaches `main` only through a reviewed pull request, so the main branch always holds a working system.
- **Each increment is verified and demonstrated.** An increment is closed only after it has been run end-to-end and shown to the team.
- **Unfinished work is carried, not hidden.** When an increment is closed with scope outstanding, the gap is recorded as a numbered backlog item and planned into a later increment.

11.2 Increment plan and delivery record

Table 3 is the increment plan and, at the same time, the record of what was actually delivered.

Table 3: Increment plan and delivery record.

Table: Increments, branches, pull requests and outcome				
Inc.	Focus	Delivered functionality	Branch / PR	Owner Outcome
1	POS core baseline	Authenticated application shell and tab navigation; login/logout; product catalogue (list, add, delete); cart with quantities and automatic total; Express server with MongoDB Product and Sale models; full REST route layer; demonstration seed endpoint; sales history; README and the GitHub workflow	<code>tanzirul/mvp-base</code> (direct commit)	(di- TanzirulIslam Delivered
2	Sales summary	Planned increment: an aggregated sales-summary endpoint and a summary dashboard page showing total and today’s revenue	<code>ruan/dashboard</code> , PR #2	Ruan Partially delivered
3	Invoice & receipt	Printable invoice module with itemised lines, subtotal, cash paid, change, grand total, and a dedicated @media print stylesheet; wired into the checkout success path in place of a browser alert	<code>shuvo/category-receipt</code> , PR #3	Shuvo Delivered

Increments (continued)				
4	Report & collaboration hardening	This Design	Project <code>tanzirul/report-lab2</code> , Report; PR #4	TanzirulDelivered Islam
<code>docs/TEAM_WORKFLOW.md</code> ; MVP screen captures; README aligned with the delivered system; report build corrected to embed outline fonts instead of bitmap fonts				

11.3 Honest statement of Increment 2

Increment 2 is reported as partially delivered rather than as complete. Pull request #2 was opened with the message “add sales summary and endpoint and dashboard page”, but the merged diff contains only route-level changes in `saleRoutes.js` and a comment cleanup in `App.jsx`; neither a summary endpoint nor a dashboard page reached `main`. Rather than reporting the increment as complete, the team records the outstanding summary widget as backlog item B-01 in Section 14.9 and schedules it for the next increment. Reporting this gap is the point of using an incremental model: an unfinished increment has a visible, nameable remainder, and the remainder does not block the delivery of the other three increments.

11.4 Figure: incremental delivery for QuickPOS

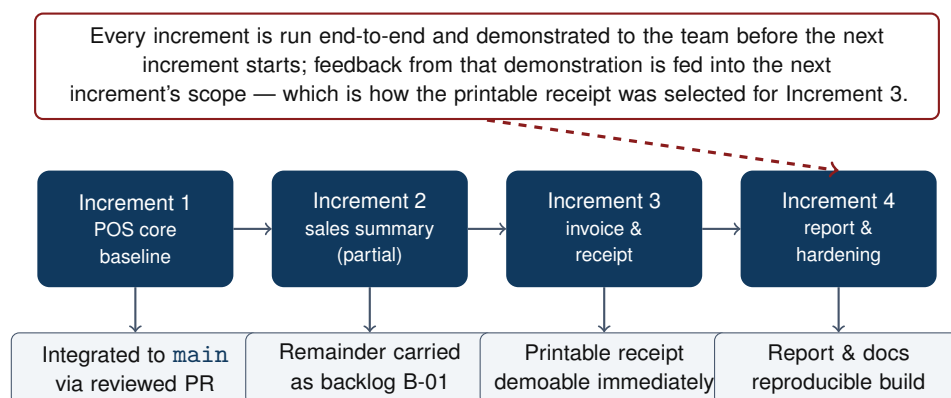


Figure 2: Incremental delivery of QuickPOS: each increment is integrated into `main` through a reviewed pull request and demonstrated before the next increment begins.

12 Justification of the Selected Process Model

- **The deliverable is defined as an MVP.** The lab asks for a small prototype that demonstrates feasibility, not for a complete system. Incremental development starts from a thin working core and grows; it does not require the complete specification to exist before any code is written, which is exactly the situation here.
- **The requirements are clear but naturally decomposable.** POS requirements are well understood and stable, so they can be planned up front. What makes them a good fit for increments is that they partition cleanly into end-to-end slices — a cart without a total is not demonstrable, and a total without cash handling is not a POS bill. Each slice fits in one increment.
- **Early and continuous client feedback.** Each increment ends in something the course teacher

can actually run. Cash handling, for example, was shown as a working button in Increment 1 and only then hardened, while the printable receipt — an addition that was not in the original plan — was pulled forward into Increment 3 after the demonstration.

- **It maps directly onto the required GitHub workflow.** One increment, one feature branch, one pull request, one merge. The process model and the collaboration requirement reinforce each other instead of competing.
- **It suits a three-member parallel team.** The three module boundaries (baseline backend and core screens, receipt, summary) map onto three branch owners who can work concurrently with minimal merge contention.
- **It fits a semester-long project.** The lab manual states that each lab builds on the previous one. An incremental model keeps the backlog open across labs, so design patterns and testing can be added as later increments on a system that is already running — no migration is needed.
- **Low blast radius.** If one increment is rejected or defective, only that slice is affected; the previously merged increments keep working. In Increment 2 this was demonstrated in practice: the incomplete dashboard affected nothing else.
- **Requirements traceability is natural.** Because every increment is small, each requirement can be linked to the pull request that delivered it, which is what Table 1 does.

13 Comparison with Alternative Process Models

13.1 Comparative summary

Comparison of candidate process models				
Criterion	Incremental	Waterfall	Spiral	Scrum
Requirement stability needed	Medium — stable but decomposable	High — must be frozen before coding	Medium	Low to medium
Feedback frequency	Every increment	Once, at the end	Every loop	Every sprint
Time to first demonstrable product	End of Increment 1 (days)	End of implementation (weeks)	After the first loop	End of Sprint 1 (days)
Documentation overhead	Light — increment plan & backlog	Heavy — full upfront specification	Heavy — risk analysis per loop	Medium — backlog, reviews, retrospectives
Change cost after delivery	Low — isolated to the next increment	Very high — rework of completed phases	Medium	Low to medium
Fit for a 3-member team on a short lab timeline	Strong	Weak	Weak	Moderate to strong

13.2 Why Waterfall was less suitable

Waterfall requires the requirement set to be frozen before design begins and delivers the system in one pass at the end. Two properties of QuickPOS rule it out. First, the deliverable is an MVP that must be demonstrable early, whereas Waterfall's first demonstrable output arrives only after implementation is complete — the opposite of what this lab asks for. Second, the requirements were

not fully frozen in advance: the printable invoice was not part of the initial plan and entered the project as Increment 3 after the team saw how weak a plain browser alert was as sale confirmation. In Waterfall that addition would have required reopening a completed design phase, which is slow and expensive for a small team.

13.3 Why Spiral was less suitable

Spiral is designed for projects with significant technical or business uncertainty, where each loop performs systematic risk analysis and prototype evaluation. QuickPOS carries very little technical risk: the hard parts (cart arithmetic, change calculation, stock deduction) are simple transactions, and the stack — React, Express, MongoDB — is familiar and well documented. Spiral’s risk-analysis machinery would have consumed a disproportionate share of a one-session lab timeline while reducing essentially none of the actual project risk. Spiral would be appropriate for this domain if, for example, the shop required offline first operation with later synchronisation, a genuinely uncertain hardware integration.

13.4 Why Agile Scrum was less suitable

Scrum delivers working software in short cycles — the same core benefit that Incremental provides — but it adds role separation, sprint ceremonies, estimation in story points and retrospectives. For a one-session academic MVP with a three-student team, stable technology and a requirement set already documented in Section 7, this ceremony adds process weight without adding feedback that the team did not already have. The team adopted the parts of Scrum that genuinely helped — a visible increment plan, a prioritised backlog, and a review before each merge — and obtained the same benefit with far less overhead.

13.5 The remaining models

- **Prototype.** Suited to projects where the user interface or the workflow is genuinely unknown and must be explored before it can be specified. QuickPOS’s workflow (a POS screen has been understood for decades) does not need exploring, and throwing away a prototype afterwards would waste the work rather than refine it.
- **RAD.** Suited to large teams building business applications from detailed, pre-existing specifications on a fixed, compressed timeline. QuickPOS is a three-member team with a small requirement set, so RAD’s component-based tooling would be disproportionate.
- **Agile XP.** Its emphasis on test-first development, pair programming and small releases is compatible with the approach, but the pair-programming discipline is not practical for three part-time students on different machines. Automated testing is planned as a later increment (backlog B-07).
- **Kanban.** A strong fit for continuous flow with prioritised demand, but there is no continuous flow to manage here: work arrives in deliberate, time-boxed increments per the lab schedule, which is precisely what Incremental models.

14 MVP Design Overview

14.1 Technology stack

Technology used for the MVP Layer Technology Version / Note

Frontend framework React with Vite React 18.3, Vite 5.4 — component-based UI with a fast development server and production build

Styling Tailwind CSS 3.4 — utility-first styling; no separate UI framework or icon package required

Frontend routing In-application page state `App.jsx` holds the active page in component state; a full router is unnecessary for three screens

Backend runtime Node.js with Express Express 4.19 — minimal REST layer, JSON body parsing and CORS enabled

Database MongoDB via Mongoose Mongoose 8.5 — `Product` and `Sale` collections with schema-level validation and defaults

Configuration dotenv `server/.env` for the database URI and port; `.env` is never committed

Version control Git and GitHub Feature branches, pull requests, reviewed merges

Reporting LaTeX `docs/Project_Design_Report.tex` compiled with `pdflatex`; outline fonts are embedded so the report stays legible on screen and in print

14.2 Architecture

QuickPOS is a three-tier client-server application. Figure 3 shows the request path for a checkout, which is the most demanding operation in the system because it reads products, validates payment, writes stock changes and inserts a sale.

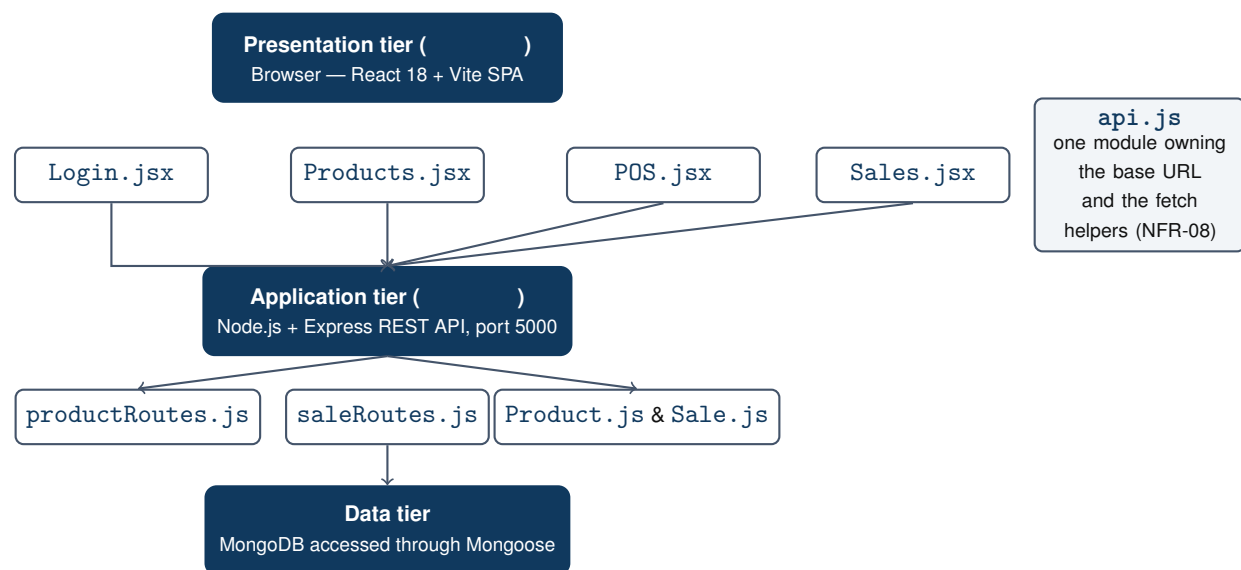


Figure 3: Three-tier architecture of the QuickPOS MVP. The checkout path is `POS.jsx` → `saleRoutes.js` → `Sale` and `Product`.

14.3 Backend interface

REST endpoints exposed by the MVP			
Method	Endpoint	Purpose	Used by
GET	<code>/api/products</code>	List all products, sorted by name	<code>POS.jsx</code> , <code>Products.jsx</code>
POST	<code>/api/products</code>	Create a product	<code>Products.jsx</code>
PUT	<code>/api/products/:id</code>	Update a product	(API only — backlog B-02)
DELETE	<code>/api/products/:id</code>	Delete a product	<code>Products.jsx</code>
GET	<code>/api/sales</code>	Most recent 50 sales, newest first	<code>Sales.jsx</code>
POST	<code>/api/sales</code>	Checkout: validates payment, decrements stock, stores the sale	<code>POS.jsx</code>
GET	<code>/api/seed</code>	Populate the catalogue with demonstration products	Demonstration setup

14.4 Module breakdown and ownership

Table: Modules, files, responsibility and owner

Module	Files	Responsibility	Owner
Authentication	<code>pages/Login.jsx</code> , <code>App.jsx</code>	Credential gate, session persistence in <code>localStorage</code> , logout, page navigation	Tanzirul Islam
Product catalogue	<code>pages/Products.jsx</code> , <code>routes/productRoutes.js</code> , <code>models/Product.js</code>	List, add and delete products; price, stock and category fields	Tanzirul Islam
Billing / POS	<code>pages/POS.jsx</code> , <code>routes/saleRoutes.js</code>	Cart with quantities, running total, checkout transaction, stock validation and deduction, sale persistence	Tanzirul Islam
Cash handling	<code>pages/POS.jsx</code> , <code>routes/saleRoutes.js</code>	Cash-received input, live change calculation, insufficient-payment rejection on both tiers	Tanzirul Islam
Invoice / receipt	<code>pages/Invoice.jsx</code>	Printable receipt modal: itemised lines, subtotal, cash paid, change, grand total, print stylesheet, print and close actions	Shuvo
Sales history	<code>pages/Sales.jsx</code>	Newest-first list of bills with total, cash paid and change; empty state message	Tanzirul Islam
Sales summary — (not delivered) dashboard		Planned aggregated revenue view; merged PR delivered route hardening only; remainder recorded as backlog B-01	Ruan
API client helper	<code>src/api.js</code>	Base URL plus <code>getJSON</code> / <code>postJSON</code> , centralising the endpoint contract	Tanzirul Islam
Seed data	<code>server/server.js</code>	Application bootstrap, MongoDB connection, <code>/api/seed</code> demonstration fixtures	Tanzirul Islam
Documentation & report	<code>docs/</code> , <code>README.md</code> , <code>screenshots/</code>	Project Design Report, team work-flow document, MVP screen captures, repository documentation	Tanzirul Islam

14.5 Component structure

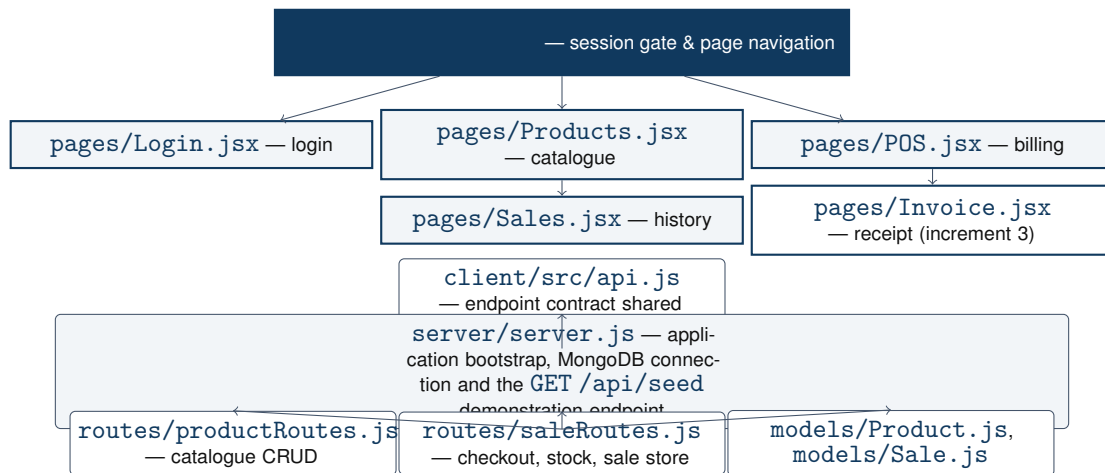


Figure 4: Component structure of the MVP. `Invoice.jsx` was added as a separate component in Increment 3 rather than being inlined into the billing screen, which keeps the billing logic readable and allows the print stylesheet to be scoped to one component.

14.6 Cash handling as a transaction

The cash-handling rule is deliberately implemented twice, on purpose:

$$\begin{aligned}
 \text{total} &= \sum (\text{unit price} \times \text{quantity}) \text{ over all cart lines} \\
 \text{change} &= \text{paidAmount} - \text{total} \\
 \text{accept the sale} &\iff \text{paidAmount} \geq \text{total} \quad \text{and} \quad \text{cart is non-empty}
 \end{aligned}$$

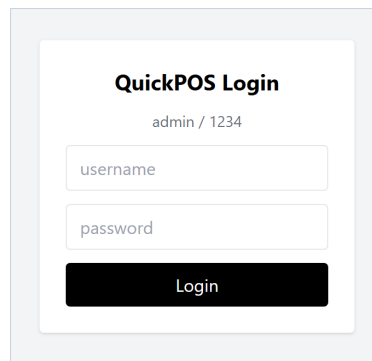
The client disables the *Checkout* button and shows a live *Change to Return* panel so the cashier gets immediate feedback (FR-11, FR-12). The server repeats the same comparison and returns HTTP 400 if it fails, before any stock is written — the client check is a usability aid, the server check is the enforcement point (NFR-10). The stock check (FR-14) runs inside the same loop that builds the bill items, and because the loop validates every line before any value is written back, a rejected sale leaves stock completely untouched (NFR-12).

14.7 Sale document

Table: Fields of the Sale document		
Field	Type	Meaning
<code>items</code>	Array of objects	The sold lines, each holding <code>productId</code> , <code>name</code> , <code>price</code> and <code>qty</code> . Name and price are copied at sale time so the bill stays historically correct even if the catalogue changes later.
<code>total</code>	Number	Bill total computed from the lines (required).
<code>paidAmount</code>	Number	Cash received from the customer; defaults to the total for exact-payment sales.
<code>change</code>	Number	Cash to return, i.e. <code>paidAmount - total</code> .
<code>date</code>	Date	Timestamp of the transaction, defaulting to the moment of creation; drives the sales-history ordering (NFR-14).

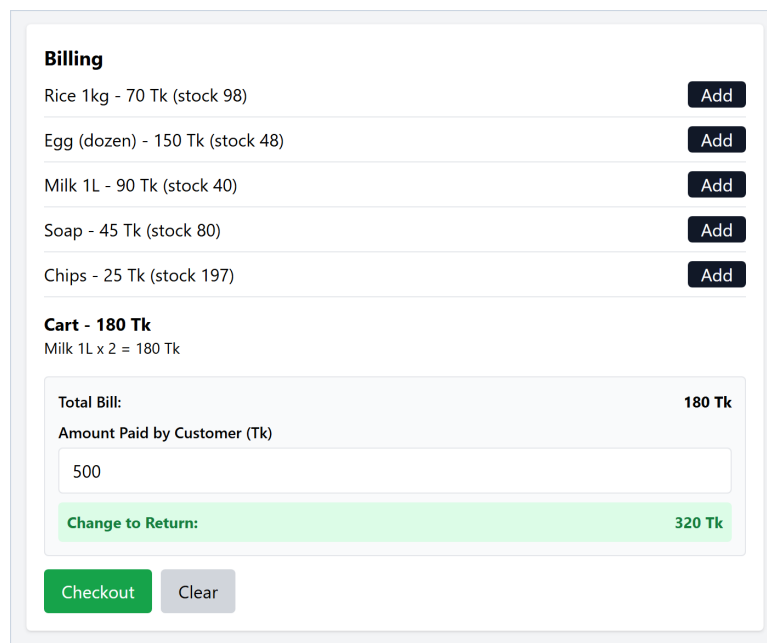
14.8 MVP demonstration screens

The following captures were taken from the running MVP. The API was populated with the demonstration products of `/api/seed`.



The image shows a login screen titled "QuickPOS Login". Below the title, it displays "admin / 1234". There are two input fields: "username" and "password". Below these fields is a black button labeled "Login".

Figure 5: US-1 — staff login gate. No billing screen is reachable without a session.



The image shows a billing screen. It has a section titled "Billing" with a list of items: "Rice 1kg - 70 Tk (stock 98)", "Egg (dozen) - 150 Tk (stock 48)", "Milk 1L - 90 Tk (stock 40)", "Soap - 45 Tk (stock 80)", and "Chips - 25 Tk (stock 197)". Each item has an "Add" button. Below this is a section titled "Cart - 180 Tk" with the text "Milk 1L x 2 = 180 Tk". There is a summary box with "Total Bill: 180 Tk" and "Amount Paid by Customer (Tk)" with a text input field containing "500". Below the input field is a green bar labeled "Change to Return: 320 Tk". At the bottom are two buttons: "Checkout" (green) and "Clear" (gray).

Figure 6: US-2 and US-3 — billing screen with a four-line bill (490 Tk), cash received of 500 Tk entered, and the *Change to Return* panel showing 10 Tk with the *Checkout* button enabled.

QuickPOS
Point of Sale System

INVOICE
#A7B8C9E0

Date & Time: Oct 1, 2026, 2:14 AM

ITEM	QTY	PRICE	AMOUNT
Milk 1L	2	90	180.00

Subtotal: 180.00 Tk
Amount Paid: 500.00 Tk
Change Returned: 320.00 Tk

Grand Total: 180.00 Tk

Thank you for your purchase! 🎉

[Print Invoice](#) [Close](#)

Figure 7: US-5 — printable invoice produced by Increment 3, showing itemised lines, subtotal 490 Tk, amount paid 500 Tk, change returned 10 Tk and the grand total.

Products

Sugar 1kg 85 60 [Add](#)

Name	Price	Stock	
Rice 1kg	70	98	Delete
Egg (dozen)	150	48	Delete
Milk 1L	90	40	Delete
Soap	45	80	Delete
Chips	25	197	Delete

Figure 8: US-4 input side — the product catalogue with the add-product form used to maintain price and stock.

Sales History

9/21/2026, 4:24:00 PM - Total: 230 Tk
 - Rice 1kg x 2 = 140 Tk
 - Milk 1L x 1 = 90 Tk
 Paid: 500 Tk Change: 270 Tk

9/21/2026, 5:02:00 PM - Total: 225 Tk
 - Egg (dozen) x 1 = 150 Tk
 - Chips x 3 = 75 Tk
 Paid: 225 Tk Change: 0 Tk

9/21/2026, 6:47:00 PM - Total: 90 Tk
 - Soap x 2 = 90 Tk
 Paid: 100 Tk Change: 10 Tk

Figure 9: US-6 — sales history with each bill's timestamp, itemised lines, total, cash paid and change returned.

14.9 Backlog carried to the next increment

The following items were identified during this lab and are planned as the next increment. They are listed here so that the boundary of the MVP is explicit.

Backlog for the next increment		
ID	Item	Origin
B-01	Sales summary dashboard: aggregated total revenue, today's revenue and number of bills, with the supporting summary endpoint	Increment 2 remainder (PR #2)
B-02	Product edit screen wired to the existing PUT /api/products/:id route	FR-06 is currently API-only
B-03	Category selection in the add-product form — the Product model and the seed data already carry a category field, but the form does not collect it	Model/UI mismatch noticed in review
B-04	Low-stock indicator and reorder threshold	Objected to by the shop owner scenario: stock is known but not flagged
B-05	Date-range filter and receipt numbering on the sales history	Recurring shop need for end-of-day reconciliation
B-06	Role separation (owner vs. cashier) and hashed credentials	Current single shared account is an MVP simplification
B-07	Extract the change calculation into a pure function and add automated unit tests, including a property test that change ≥ 0	Preparation for the testing lab

15 GitHub Collaboration Evidence

15.1 Repository

Repository link: <https://github.com/TanzirulIslam22/QuickPOS>

Branches: main (integration), tanzirul/mvp-base, ruan/dashboard, shuvo/category-receipt, tanzirul/report-lab2

Repository contributors (GitHub): TanzirulIslam22, mahdi-ruan, Hi-There-shuvo

15.2 Repository directory structure

Repository layout

QuickPOS/	
-- client/	React + Vite frontend (Increment 1)
-- src/	
-- api.js	base URL + getJSON / postJSON helpers
-- App.jsx	app shell, session gate, page navigation
-- main.jsx	React entry point
-- index.css	Tailwind entry point
'-- pages/	
-- Login.jsx	staff login screen
-- Products.jsx	product catalogue (list / add / delete)
-- POS.jsx	billing: cart, total, cash, checkout
-- Sales.jsx	sales history
'-- Invoice.jsx	printable receipt modal (Increment 3)
-- index.html	Vite entry template
-- vite.config.js	dev server + build config
-- tailwind.config.js	Tailwind content paths
-- postcss.config.js	PostCSS pipeline

```
|  '-- package.json
|-- server/           Express + MongoDB backend (Increment 1)
|  |-- server.js      app bootstrap, DB connect, /api/seed
|  |-- models/
|  |  |-- Product.js  name, price, stock, category
|  |  '-- Sale.js     items, total, paidAmount, change, date
|  |-- routes/
|  |  |-- productRoutes.js  GET/POST/PUT/DELETE /api/products
|  |  '-- saleRoutes.js     GET/POST /api/sales (NFR-10 check)
|  |-- .env.example    documented configuration template
|  '-- package.json
|-- docs/
|  |-- Project_Design_Report.tex  LaTeX source of this report
|  |-- Requirement_Report.pdf     this report (compiled)
|  '-- TEAM_WORKFLOW.md          branching, PR and review rules
|-- assets/                     design assets used by the report
|-- screenshots/                MVP screen captures (Section 8.8)
|-- lab_2.pdf                   lab manual
'-- README.md                   project overview, API table, team
```

Note on the suggested folder

The lab manual's suggested layout lists a single `src/` directory. QuickPOS keeps its source in two top-level modules, `client/src` (frontend) and `server` (backend), so that the two separately buildable and separately deployable layers stay separated. This is the same separation the manual asks for, expressed at the deployment boundary rather than inside one folder.

15.3 Branching strategy

Branching strategy		
Branch	Owner	Purpose
main	All (integration only)	Stable, reviewed, integrated codebase; the only branch that is deployed or demonstrated
tanzirul/mvp-base	Tanzirul Islam	Increment 1 — POS core baseline
ruan/dashboard	Ruan	Increment 2 — sales summary (partially delivered)
shuvo/category-receipt	Shuvo	Increment 3 — printable invoice / receipt
tanzirul/report-lab2	Tanzirul Islam	Increment 4 — report, workflow documentation and screenshots

15.4 Pull requests

Table: Pull requests raised against main

PR	Head branch	Title	Diff	State / outcome
#1	tanzirul/mvp-base	docs: add QuickPOS team information	+28/ − 15 1 file	Open — superseded by PR #4, which carries the final documentation set
#2	ruan/dashboard	feature: add sales summary and endpoint and dashboard page	+1/ − 5 2 files	Merged 21 Sep 2026 — partial (see Section 11.3)
#3	shuvo/category-receipt	Feature - Invoice function added	+204/ − 45 2 files	Merged 21 Sep 2026 — complete
#4	tanzirul/report-lab2	Increment 4: project design report, workflow docs and MVP screenshots	docs set	Merged into main — complete

15.5 Commit history

The history below is the actual output of `git log -graph` on [main](#). The merge commits make each increment visible as a separate, reviewable unit, and the author line of each non-merge commit identifies the member who wrote it.

`git log -graph main`

```
* 2cbd99a Merge pull request #3 from TanzirulIslam22/shuvo/category-receipt
|\
| * da55ea0 Feature - Invoice function added (Hi-There-shuvo)
|/
* ce4f8a1 Merge pull request #2 from TanzirulIslam22/ruan/dashboard
|\
| * a0f9b10 feature: add sales summary and endpoint (mahdi-ruan)
| |         and dashboard page
|/
* 64d8f4d feat: initial QuickPOS MVP (login, (TanzirulIslam22)
|         products, billing, sales, cash handler)
```

Contribution by branch (`git log -format, per branch`)

tanzirul/mvp-base	64d8f4d	Tanzirul Islam	initial MVP baseline
ruan/dashboard	a0f9b10	Ruan	sales summary increment (partial)
shuvo/category-receipt	da55ea0	Shuvo	invoice / receipt increment

15.6 Pull request review protocol

- A feature branch is opened from an up-to-date [main](#) (`git pull origin main` first), so that branches stay short-lived and conflicts are rare.
- The pull request description states the increment number, the requirements it satisfies, and anything left unfinished.
- At least one other team member reviews and approves before the merge; review comments are resolved rather than dismissed.
- Conflicts are resolved locally by the pull request author, re-tested, and re-pushed for a final review pass.
- No direct pushes to [main](#). Every change lands through a reviewed pull request.

15.7 Collaboration issues encountered

- **Branch and title drift.** Two branch names promised more than their increments delivered (`ruan/dashboard`, `shuvo/category-receipt`). The team resolved this by recording the *actual* merged diff per pull request in Table 3 and by numbering the remaining work as backlog items, rather than letting the branch name stand as the description.
- **Local branch drift.** Working from a feature branch that had fallen behind `main` left changes staged in the index that no longer matched any branch. Resolution: `git fetch` and fast-forward `main` to `origin/main` before starting new work, and never leave unrelated changes staged.

16 Challenges Encountered and Engineering Mitigations

16.1 C-1: The first report build rendered faint, low-contrast text

The first version of this report was compiled without outline fonts being available, so pdfTeX fell back to 1-bit bitmap (PK) fonts. The result was text drawn from very low resolution bitmaps: thin, jagged strokes with poor contrast that were difficult to read on screen and poor in print.

Mitigation. The document now loads an explicit Type 1 outline family (Latin Modern for body text, Helvetica for headings) and applies a fixed, deliberately dark colour scheme: body text is near-black rather than grey, headings use a dark navy and a dark teal, and table rules and text stay dark. The build was verified afterwards by inspecting the font objects embedded in the output PDF and confirming that they are Type 1 outlines with no Type 3 bitmap fonts present. This is the change requested after reviewing the earlier draft.

16.2 C-2: Client-side payment validation is not enforcement

An early concern was that a sale could be recorded with insufficient cash, since the cash amount is entered in the browser.

Mitigation. Validation is deliberately duplicated. The client disables the *Checkout* button and marks the amount as insufficient, which is a usability measure; the server independently recomputes the total, rejects `paidAmount < total` with HTTP 400 *before* writing anything, and is the only component whose check counts (NFR-10).

16.3 C-3: A sale could have partially decremented stock

Validating each cart line while decrementing it would leave stock corrupted if a later line failed — for example, if a customer asked for 3 of an item that only had 1.

Mitigation. The route first builds the full list of bill items, validating stock for every line, and only then performs the writes; a rejected sale therefore leaves stock exactly as it was (NFR-12, FR-14). The requirement is captured as a user story with an explicit acceptance criterion (US-4).

16.4 C-4: An increment was merged with less delivered than its branch name promised

Pull request #2 was named and titled as a dashboard increment but merged only route-level changes, leaving the actual summary widget undelivered.

Mitigation. Three changes followed. First, the report states the delivered scope of each increment from the merged diff rather than from the branch name (Section 3). Second, the remainder is recorded as backlog item B-01 with a named owner. Third, the team’s review protocol now requires every pull request description to declare unfinished work explicitly, so an incomplete increment is visible in the repository itself and not only in a report.

16.5 C-5: Environment differences between machines

Each member develops on a different machine, and the database connection string differs between them.

Mitigation. A documented `server/.env.example` is committed, the real `.env` is listed in `.gitignore` so no connection string can be committed by accident, and the port is read from the environment with a default, so a member's local port choice cannot break the frontend contract.

16.6 C-6: The billing component was growing beyond a single responsibility

Adding the receipt to the billing screen pushed `POS.jsx` towards handling cart state, payment validation and receipt rendering in one place.

Mitigation. The receipt was extracted into its own `Invoice.jsx` component that receives the completed sale and a close callback as its only interface. `POS` keeps ownership of the cart and payment flow, the invoice owns presentation and printing, and the print stylesheet is scoped to the modal via an `@media print` block and a no-print utility class (NFR-15).

17 Conclusion

In Lab 2 the team analysed a real retail problem, documented eighteen functional and fifteen non-functional requirements with traceability to the delivered code, selected and justified the Incremental Development process model, and delivered a working MVP of a super shop POS system: authenticated billing, a maintained product catalogue, cart and total computation, cash handling with change calculation, automatic stock validation and deduction, a printable invoice, and a persistent sales history — built with React, Vite, Tailwind CSS, Express and MongoDB.

The Incremental model proved practical rather than merely formal. Four increments were planned and delivered as four reviewable units on four feature branches, and the model made the one incomplete increment visible: instead of Increment 2 disappearing into a vague commit message, its remainder is a named backlog item with an owner. Collaboration followed the same shape — no direct pushes to the integration branch, every change landed through a reviewed pull request, and each member's contribution is traceable in the commit history and in the ownership tables of this report.

As the first milestone of a semester-long project, QuickPOS was deliberately built as small, replaceable modules with a single API client module and a single route layer, so that the design-pattern and testing labs can add to the system incrementally rather than requiring a rewrite. The immediate next increment is defined in Section 14.9: the sales summary dashboard, the product edit screen, category selection, a low-stock indicator, date-range reporting, role separation, and unit tests for the cash-handling arithmetic.

References

- [1] Department of Computer Science & Engineering, RUET. *CSE 3206 — Software Engineering Sessional, Lab 2: Software Process Models, Requirement Analysis & MVP Development* — lab manual and assessment rubric.
- [2] Pressman, R. S. and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill. (Source of the process-model definitions used in Sections 11 and 12.)